Chapter 25

AES Accelerator (AES)

25.1 Introduction

ESP32-P4 integrates an Advanced Encryption Standard (AES) accelerator, which is a hardware device that speeds up computation using AES algorithm significantly, compared to AES algorithms implemented solely in software. The AES accelerator integrated in ESP32-P4 has two working modes, which are Typical AES and DMA-AES.

25.2 Features

The following functionality is supported:

- Typical AES working mode
  - AES-128/AES-256 encryption and decryption
- DMA-AES working mode
  - AES-128/AES-256 encryption and decryption
  - Block cipher mode
    - ECB (Electronic Codebook)
    - CBC (Cipher Block Chaining)
    - OFB (Output Feedback)
    - CTR (Counter)
    - CFB8 (8-bit Cipher Feedback)
    - CFB128 (128-bit Cipher Feedback)
  - GCM (Galois/Counter Mode)
  - Interrupt on completion of computation

25.3 Clock and Reset

The AES accelerator is activated by setting the HP_SYS_CLKRST_CRYPTO_AES_CLK_EN bit in the HP_SYS_CLKRST_PERI_CLK_CTRL25_REG register and clearing the HP_SYS_CLKRST_RST_EN_AES bit in the HP_SYS_CLKRST_HP_RST_EN2_REG register. Besides, due to resource reuse between cryptography accelerator modules, users also need to additionally clear the HP_SYS_CLKRST_RST_EN_DS bit and the HP_SYS_CLKRST_RST_EN_KM bit in the HP_SYS_CLKRST_HP_RST_EN2_REG register.

25.4 AES Working Modes

The AES accelerator integrated in ESP32-P4 has two working modes, which are Typical AES and DMA-AES.

- Typical AES Working Mode:
  - Supports encryption and decryption using cryptographic keys of 128 and 256 bits, specified in NIST FIPS 197.

  In this working mode, the plaintext and ciphertext is written and read via CPU directly.
- DMA-AES Working Mode:
  - Supports encryption and decryption using cryptographic keys of 128 and 256 bits, specified in NIST FIPS 197;
  - Supports block cipher modes ECB, CBC, OFB, CTR, CFB8, and CFB128 under NIST SP 800-38A.
  - Supports GCM under NIST SP 800-38D.

  In this working mode, the plaintext and ciphertext are written and read via DMA. An interrupt will be generated when operation completes.

Users can choose the working mode for AES accelerator by configuring the AES_DMA_ENABLE_REG register according to Table 25.4-1 below.

Table 25.4-1. AES Accelerator Working Mode

AES_DMA_ENABLE_REG	Working Mode
0	Typical AES
1	DMA-AES

Users can choose the length of cryptographic keys and encryption/decryption by configuring the AES_MODE_REG register according to Table 25.4-2 below.

Table 25.4-2. Key Length and Encryption/Decryption

AES_MODE_REG[2:0]	Key Length and Encryption/Decryption
0	AES-128 encryption
1	reserved
2	AES-256 encryption
3	reserved
4	AES-128 decryption
5	reserved
6	AES-256 decryption
7	reserved

For a detailed introduction to these two working modes, please refer to Section 25.5 and Section 25.6 below.

Notice:

ESP32-P4's RSA Digital Signature Peripheral (RSA_DS) module will call the AES accelerator. Therefore, users cannot access the AES accelerator when RSA Digital Signature Peripheral (RSA_DS) module is working.

25.5 Typical AES Working Mode

In the Typical AES working mode, users can check the working status of the AES accelerator by inquiring the AES_STATE_REG register and comparing the return value against the Table 25.5-1 below.

Table 25.5-1. Working Status under Typical AES Working Mode

AES_STATE_REG	Status	Description
0	IDLE	The AES accelerator is idle or completed operation.
1	WORK	The AES accelerator is in the middle of an operation.

25.5.1 Key, Plaintext, and Ciphertext

The encryption or decryption key is stored in AES_KEY_*n*_REG, which is a set of eight 32-bit registers.

- For AES-128 encryption or decryption, the 128-bit key is stored in AES_KEY_0_REG ~ AES_KEY_3_REG.
- For AES-256 encryption or decryption, the 256-bit key is stored in AES_KEY_0_REG ~ AES_KEY_7_REG.

The plaintext and ciphertext are stored in AES_TEXT_IN_*m*_REG and AES_TEXT_OUT_*m*_REG, which are two sets of four 32-bit registers.

- For AES-128 or AES-256 encryption, the AES_TEXT_IN_*m*_REG registers are initialized with plaintext. Then, the AES accelerator stores the ciphertext into AES_TEXT_OUT_*m*_REG after operation.
- For AES-128 or AES-256 decryption, the AES_TEXT_IN_*m*_REG registers are initialized with ciphertext. Then, the AES accelerator stores the plaintext into AES_TEXT_OUT_*m*_REG after operation.

25.5.2 Endianness

Text Endianness

In Typical AES working mode, the AES accelerator uses cryptographic keys to encrypt and decrypt data in blocks of 128 bits. When filling data into AES_TEXT_IN_*m*_REG register or reading result from AES_TEXT_OUT_*m*_REG registers, users should follow the text endianness type specified in Table 25.5-2.

Table 25.5-2. Text Endianness Type for Typical AES

Plaintext/Ciphertext					
State[1]		c[2]			
		0	1	2	3
r	0	AES_TEXT_*x*_0_REG[7:0]	AES_TEXT_*x*_1_REG[7:0]	AES_TEXT_*x*_2_REG[7:0]	AES_TEXT_*x*_3_REG[7:0]
	1	AES_TEXT_*x*_0_REG[15:8]	AES_TEXT_*x*_1_REG[15:8]	AES_TEXT_*x*_2_REG[15:8]	AES_TEXT_*x*_3_REG[15:8]
	2	AES_TEXT_*x*_0_REG[23:16]	AES_TEXT_*x*_1_REG[23:16]	AES_TEXT_*x*_2_REG[23:16]	AES_TEXT_*x*_3_REG[23:16]

Table 25.5-2. Text Endianness Type for Typical AES

Plaintext/Ciphertext					
	3	AES_TEXT_*x*_0_REG[31:24]	AES_TEXT_*x*_1_REG[31:24]	AES_TEXT_*x*_2_REG[31:24]	AES_TEXT_*x*_3_REG[31:24]

[1] The definition of "State (including c and r)" is described in Section 3.4 *The State* in NIST FIPS 197.

[2] Where *x* = IN or OUT.

Key Endianness

In Typical AES working mode, when filling keys into AES_KEY_*m*_REG registers, users should follow the key endianness type specified in Table 25.5-3 and Table 25.5-4.

Table 25.5-3. Key Endianness Type for AES-128 Encryption and Decryption

Bit[1]	w[0]	w[1]	w[2]	w[3][2]
[31:24]	AES_KEY_0_REG[7:0]	AES_KEY_1_REG[7:0]	AES_KEY_2_REG[7:0]	AES_KEY_3_REG[7:0]
[23:16]	AES_KEY_0_REG[15:8]	AES_KEY_1_REG[15:8]	AES_KEY_2_REG[15:8]	AES_KEY_3_REG[15:8]
[15:8]	AES_KEY_0_REG[23:16]	AES_KEY_1_REG[23:16]	AES_KEY_2_REG[23:16]	AES_KEY_3_REG[23:16]
[7:0]	AES_KEY_0_REG[31:24]	AES_KEY_1_REG[31:24]	AES_KEY_2_REG[31:24]	AES_KEY_3_REG[31:24]

[1] Column "Bit" specifies the bytes of each word stored in w[0] ~ w[3].

[2] w[0] ~ w[3] are "the first Nk words of the expanded key" as specified in Section 5.2 *Key Expansion* in NIST FIPS 197.

Table 25.5-4. Key Endianness Type for AES-256 Encryption and Decryption

Bit [1]	w[0]	w[1]	w[2]	w[3]	w[4]	w[5]	w[6]	w[7] [2]
[31:24]	AES_KEY_0_REG[7:0]	AES_KEY_1_REG[7:0]	AES_KEY_2_REG[7:0]	AES_KEY_3_REG[7:0]	AES_KEY_4_REG[7:0]	AES_KEY_5_REG[7:0]	AES_KEY_6_REG[7:0]	AES_KEY_7_REG[7:0]
[23:16]	AES_KEY_0_REG[15:8]	AES_KEY_1_REG[15:8]	AES_KEY_2_REG[15:8]	AES_KEY_3_REG[15:8]	AES_KEY_4_REG[15:8]	AES_KEY_5_REG[15:8]	AES_KEY_6_REG[15:8]	AES_KEY_7_REG[15:8]
[15:8]	AES_KEY_0_REG[23:16]	AES_KEY_1_REG[23:16]	AES_KEY_2_REG[23:16]	AES_KEY_3_REG[23:16]	AES_KEY_4_REG[23:16]	AES_KEY_5_REG[23:16]	AES_KEY_6_REG[23:16]	AES_KEY_7_REG[23:16]
[7:0]	AES_KEY_0_REG[31:24]	AES_KEY_1_REG[31:24]	AES_KEY_2_REG[31:24]	AES_KEY_3_REG[31:24]	AES_KEY_4_REG[31:24]	AES_KEY_5_REG[31:24]	AES_KEY_6_REG[31:24]	AES_KEY_7_REG[31:24]

[1] Column "Bit" specifies the bytes of each word stored in w[0] ~ w[7].

[2] w[0] ~ w[7] are "the first Nk words of the expanded key" as specified in Chapter 5.2 *Key Expansion* in NIST FIPS 197.

25.5.3 Operation Process

Single Operation

1. Write 0 to the AES_DMA_ENABLE_REG register.
2. Initialize registers AES_MODE_REG, AES_KEY_*n*_REG, and AES_TEXT_IN_*m*_REG.
3. Start operation by writing 1 to the AES_TRIGGER_REG register.
4. Wait till the content of the AES_STATE_REG register becomes 0, which indicates the operation is completed.
5. Read results from the AES_TEXT_OUT_*m*_REG register.

Consecutive Operations

In consecutive operations, primarily the input AES_TEXT_IN_*m*_REG and output AES_TEXT_OUT_*m*_REG registers (*m*: 0-3) are being written and read, while the content of AES_DMA_ENABLE_REG, AES_MODE_REG, and AES_KEY_*n*_REG is kept unchanged. Therefore, the initialization can be simplified during the consecutive operation.

1. Write 0 to the AES_DMA_ENABLE_REG register before starting the first operation.
2. Initialize registers AES_MODE_REG and AES_KEY_*n*_REG before starting the first operation.
3. Update the content of AES_TEXT_IN_*m*_REG.
4. Start operation by writing 1 to the AES_TRIGGER_REG register.
5. Wait till the content of the AES_STATE_REG register becomes 0, which indicates the operation completes.
6. Read results from the AES_TEXT_OUT_*m*_REG register, and return to Step 3 to continue the next operation.

25.6 DMA-AES Working Mode

In the DMA-AES working mode, the AES accelerator supports six block cipher modes: ECB, CBC, OFB, CTR, CFB8, and CFB128. Users can choose the block cipher mode by configuring the AES_BLOCK_MODE_REG register according to Table 25.6-1 below.

Table 25.6-1. Block Cipher Mode

AES_BLOCK_MODE_REG[2:0]	Block Cipher Mode
0	ECB (Electronic Codebook)
1	CBC (Cipher Block Chaining)
2	OFB (Output Feedback)
3	CTR (Counter)
4	CFB8 (8-bit Cipher Feedback)
5	CFB128 (128-bit Cipher Feedback)
6	GCM (Galois/Counter Mode)
7	reserved

Users can check the working status of the AES accelerator by inquiring the AES_STATE_REG register and comparing the return value against the Table 25.6-2 below.

Table 25.6-2. Working Status under DMA-AES Working mode

AES_STATE_REG[1:0]	Status	Description
0	IDLE	The AES accelerator is idle.
1	WORK	The AES accelerator is in the middle of an operation.
2	DONE	The AES accelerator completed operations.

When working in the DMA-AES working mode, the AES accelerator supports interrupt on the completion of computation. To enable this function, write 1 to the AES_INT_ENA_REG register. By default, the interrupt function is disabled. Also, note that the interrupt should be cleared by software after use.

25.6.1 Key, Plaintext, and Ciphertext

Block Operation

During the block operations, the AES accelerator reads source data from DMA, and writes result data to DMA after the computation.

- For encryption, DMA reads plaintext from memory, then passes it to AES as source data. After computation, AES passes ciphertext as result data back to DMA to write into memory.
- For decryption, DMA reads ciphertext from memory, then passes it to AES as source data. After computation, AES passes plaintext as result data back to DMA to write into memory.

During block operations, the lengths of the source data and result data are the same. The total computation time is reduced because the DMA data operation and AES computation can happen concurrently.

The length of source data for AES accelerator under DMA-AES working mode must be 128 bits or the integral multiples of 128 bits. Otherwise, trailing zeros will be added to the original source data, so the length of source data equals to the nearest integral multiples of 128 bits. Please see details in Table 25.6-3 below.

Table 25.6-3. TEXT-PADDING

Function : TEXT-PADDING( )
Input : X, bit string.
Output : Y = **TEXT-PADDING**(X), whose length is the nearest integral multiples of 128 bits.
Steps Let us assume that X is a data-stream that can be split into *n* parts as following: $X = X_1 \| \| X_2 \| \| \cdots \| \| X_{n-1} \| \| X_n$ Here, the lengths of $X_1, X_2, \cdots, X_{n-1}$ all equal to 128 bits, and the length of X_n is *t* (0<=*t*<=127). If $t = 0$, then **TEXT-PADDING**(X) = X; If $0 < t <= 127$, define a 128-bit block, X_n^*, and let $X_n^* = X_n \| \| 0^{128-t}$, then **TEXT-PADDING**($X$) = $X_1 \| \| X_2 \| \| \cdots \| \| X_{n-1} \| \| X_n^*$ = $X \| \| 0^{128-t}$

25.6.2 Endianness

Under the DMA-AES working mode, the transmission of source data and result data for AES accelerator is solely controlled by DMA. Therefore, the AES accelerator cannot control the Endianness of the source data and result data, but does have requirement on how these data should be stored in memory and on the length of the data.

For example, let us assume DMA needs to write the following data into memory at address 0x0280.

- Data represented in hexadecimal:
  - 000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F20
- Data Length:
  - Equals to 2 blocks.

Then, this data will be stored in memory as shown in Table 25.6-4 below.

Table 25.6-4. Text Endianness for DMA-AES

Address	Byte	Address	Byte	Address	Byte	Address	Byte
0x0280	0x01	0x0281	0x02	0x0282	0x03	0x0283	0x04
0x0284	0x05	0x0285	0x06	0x0286	0x07	0x0287	0x08
0x0288	0x09	0x0289	0x0A	0x028A	0x0B	0x028B	0x0C
0x028C	0x0D	0x028D	0x0E	0x028E	0x0F	0x028F	0x10
0x0290	0x11	0x0291	0x12	0x0292	0x13	0x0293	0x14
0x0294	0x15	0x0295	0x16	0x0296	0x17	0x0297	0x18
0x0298	0x19	0x0299	0x1A	0x029A	0x1B	0x029B	0x1C
0x029C	0x1D	0x029D	0x1E	0x029E	0x1F	0x029F	0x20

25.6.3 Standard Incrementing Function

AES accelerator provides two Standard Incrementing Functions for the CTR block operation, which are INC_{32} and INC_{128} Standard Incrementing Functions. By setting the AES_INC_SEL_REG register to 0 or 1, users can choose the INC_{32} or INC_{128} functions respectively. For details on the Standard Incrementing Function, please see Chapter B.1 *The Standard Incrementing Function* in NIST SP 800-38A.

25.6.4 Block Number

Register AES_BLOCK_NUM_REG stores the Block Number of plaintext P or ciphertext C. The length of this register equals to length(**TEXT-PADDING**(P))/128 or length(**TEXT-PADDING**(C))/128. The AES accelerator only uses this register when working in the DMA-AES mode.

25.6.5 Initialization Vector

AES_IV_MEM is a 16-byte memory, which is only available for AES accelerator working in block operations. For CBC/OFB/CFB8/CFB128 operations, the AES_IV_MEM memory stores the Initialization Vector (IV). For the CTR operation, the AES_IV_MEM memory stores the Initial Counter Block (ICB).

Both IV and ICB are 128-bit strings, which can be divided into Byte0, Byte1, Byte2 $\cdots$ Byte15 (from left to right). AES_IV_MEM stores data following the Endianness pattern presented in Table 25.6-4, i.e., the most significant (i.e., left-most) byte Byte0 is stored at the lowest address while the least significant (i.e., right-most) byte Byte15 at the highest address.

For more details on IV and ICB, please refer to NIST SP 800-38A.

25.6.6 Block Operation Process

1. Select one of DMA channels to connect with AES, configure the DMA linked list, and then start DMA. For details, please refer to Chapter 4 *GDMA Controller (GDMA-AHB, GDMA-AXI)*.
2. Initialize the AES accelerator-related registers:
   - Write 1 to the AES_DMA_ENABLE_REG register.
   - Configure the AES_INT_ENA_REG register to enable or disable the interrupt function.
   - Initialize registers AES_MODE_REG and AES_KEY_*n*_REG.
   - Select block cipher mode by configuring the AES_BLOCK_MODE_REG register. For details, see Table 25.6-1.
   - Initialize the AES_BLOCK_NUM_REG register. For details, see Section 25.6.4.
   - Initialize the AES_INC_SEL_REG register (only needed when AES accelerator is working under CTR block operation).
   - Initialize the AES_IV_MEM memory (This is always needed except for ECB block operation).
3. Start operation by writing 1 to the AES_TRIGGER_REG register.
4. Wait for the completion of computation, which happens when the content of AES_STATE_REG becomes 2 or the AES interrupt occurs.

5. Check if DMA completes data transmission from AES to memory. At this time, DMA had already written the result data in memory, which can be accessed directly. For details on DMA, please refer to Chapter 4 *GDMA Controller (GDMA-AHB, GDMA-AXI)*.
6. Clear interrupt by writing 1 to the AES_INT_CLR_REG register, if any AES interrupt occurred during the computation.
7. Release the AES accelerator by writing 1 to the AES_DMA_EXIT_REG register. After this, the content of the AES_STATE_REG register becomes 0. Note that, you can release DMA earlier, but only after Step 4 is completed.

25.6.7 GCM Operation Process

1. Configure DMA chain and start DMA. For details on DMA, please refer to Chapter 4 *GDMA Controller (GDMA-AHB, GDMA-AXI)*.
2. Initialize the AES accelerator-related registers:
   - Write 1 to the AES_DMA_ENABLE_REG register.
   - Configure the AES_INT_ENA_REG register to enable or disable the interrupt function.
   - Initialize registers AES_MODE_REG and AES_KEY_*n*_REG.
   - Write 6 to the AES_BLOCK_MODE_REG register.
   - Initialize the AES_BLOCK_NUM_REG register. Details about this register are described in Section 25.6.4 *Block Number*.
   - Initialize the AES_AAD_BLOCK_NUM_REG register. Details about this register are described in Section 25.7.4 *AAD Block Number*.
   - Initialize the AES_REMAINDER_BIT_NUM_REG register. Details about this register is described in Section 25.7.5 *Number of Effective Bits of Incomplete Blocks*.
3. Start operation by writing 1 to the AES_TRIGGER_REG register.
4. Wait for the completion of computation, which happens when the content of AES_STATE_REG becomes 2. For details on the working status of AES Accelerator, please refer to Table 25.6-2 *Working Status under DMA-AES Working mode*. At this step, no interrupt occurs.
5. Obtain the H value from the AES_H_MEM memory.
6. Generate J_0 and write it to the AES_J0_MEM memory.
7. Continue operating by writing 1 to the AES_CONTINUE_OP_REG register.
8. Wait for the completion of computation, which happens when the content of AES_STATE_REG becomes 2 or the AES interrupt occurs. For details on the working status of AES Accelerator, please refer to Table 25.6-2 *Working Status under DMA-AES Working mode*.
9. Obtain T_0 by reading AES_T0_MEM.
10. Check if DMA completes data transmission from AES to memory. At this time, DMA had already written the result data in memory, which can be accessed directly. For details on DMA, please refer to Chapter 4 *GDMA Controller (GDMA-AHB, GDMA-AXI)*.

11. Clear interrupt by writing 1 to the AES_INT_CLR_REG register, if any AES interrupt occurred during the computation.
12. Exit DMA by writing 1 to the AES_DMA_EXIT_REG register. After this, the content of the AES_STATE_REG becomes 0. Note that, you can exit DMA earlier, but only after Step 8 is completed.

25.7 GCM Algorithm

ESP32-P4's AES accelerator fully supports GCM Algorithm. In reality, the AAD, C and P that are longer than 2^{32}-1 bits are seldom used. Therefore, we specify that the length of AAD, C and P should be no longer than 2^{32}-1 here. Figure 25.7-1 below demonstrates how GCM encryption is implemented in the AES Accelerator of ESP32-P4.

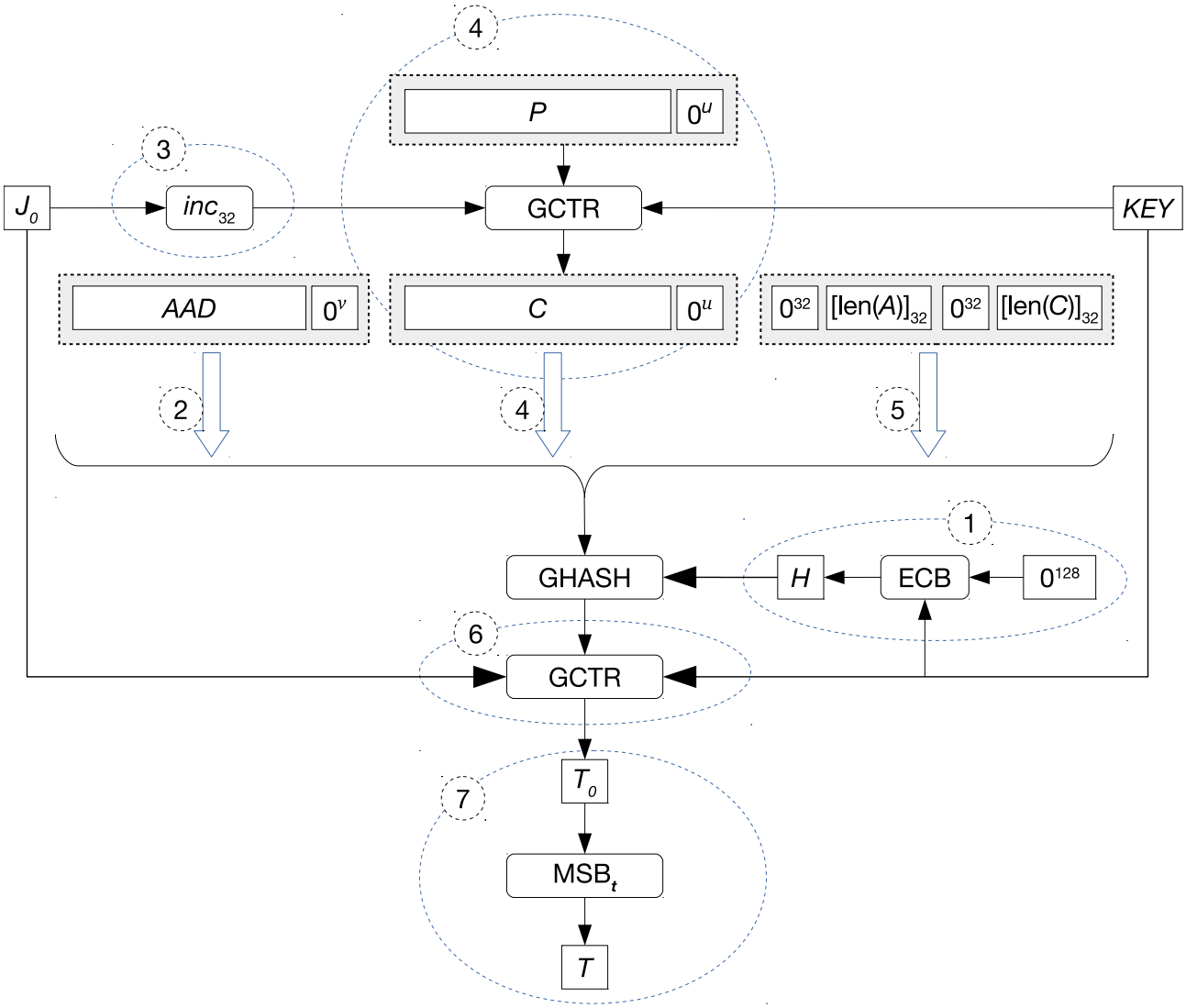


Figure 25.7-1. GCM Encryption Process

GCM encryption is implemented as follows:

1. Hardware executes the ECB Algorithm to obtain the Hash subkey H, which is needed in the Hash computation.
2. Hardware executes the GHASH Algorithm to perform Hash computation with the padded AAD.

3. Hardware gets ready for CTR encryption by obtaining the result of applying Standard Incrementing Function INC_{32} to J_0.
4. Hardware executes the GCTR Algorithm to encrypt the padded plaintext P, then executes the GHASH Algorithm to perform Hash computation on the padded ciphertext C.
5. Hardware executes the GHASH Algorithm to perform Hash computation on AAD Blocks, obtaining a 128-bit Hash result.
6. Hardware executes the GCTR Algorithm to encrypt J_0, obtaining T_0.
7. Software obtains the result T_0 from hardware, and executes MSB_t Algorithm to obtain the final authentication tag T.

The only difference between GCM decryption and GCM encryption lies in Step 4 in Figure 25.7-1. To be more specific, instead of executing GCTR Algorithm to encrypt the padded plaintext, the AES Accelerator executes the same algorithm to decrypt the padded ciphertext in GCM decryption. For details, please see NIST SP 800-38D.

25.7.1 Hash Subkey

During GCM operation, the Hash subkey H is a 128-bit value computed by hardware, which is demonstrated in Step 1 in Figure 25.7-1. Also you can find more information about Hash subkey at "Step 1. Let H = $\text{CIPH}_K(0^{128})$" in Chapter 7 GCM Specification of NIST SP 800-38D.

The Hash subkey H is stored in the AES_H_MEM memory. Just like other endianness, its most significant (i.e., left-most) byte Byte0 is stored at the lowest address in the memory while least significant (i.e., right-most) byte Byte15 at the highest address. For details, see Table 25.5-2.

25.7.2 J_0

J_0 is a 128-bit value computed by hardware, which is required during Step 3 and Step 6 in Figure 25.7-1. For details on the generation of J_0, please see Chapter 7 GCM Specification in NIST SP 800-38D.

J_0 is stored in the AES_J0_MEM memory. Just like other endianness, its most significant (i.e., left-most) byte Byte0 is stored at the lowest address in the memory while least significant (i.e., right-most) byte Byte15 at the highest address. For details, see Table 25.5-2.

25.7.3 Authentication Tag

Authentication Tag (Tag for short) is one of the key results of GCM computation, which is demonstrated in Step 7 of Figure 25.7-1. The value of the Tag is determined by its length t ($1 <= t <= 128$):

- When $t = 128$, the value of Tag equals to T_0, a 128-bit string that is stored in the AES_T0_MEM. Just like other endianness, its most significant (i.e., left-most) byte Byte0 is stored at the lowest address in the memory while least significant (i.e., right-most) byte Byte15 at the highest address. For details, see Table 25.5-2.
- When $1 <= t < 128$, the value of Tag equals to the t most significant (i.e., left-most) bits of T_0. In this case, Tag is represented as $\text{MSB}_t(T_0)$, which returns the t most significant bits of T_0. For example, MSB_4(111011010) = 1110 and MSB_5(11010011010) = 11010. For details on the $\text{MSB}_t(\,)$ function, please refer to Chapter 6 Mathematical Components of GCM in NIST SP 800-38D.

25.7.4 AAD Block Number

Register AES_AAD_BLOCK_NUM_REG stores the Block Number of Additional Authenticated Data (AAD). The length of this register equals to (**TEXT-PADDING**(AAD) length)/128. AES Accelerator only uses this register when working in the DMA-AES mode.

25.7.5 Number of Effective Bits of Incomplete Blocks

Register AES_REMAINDER_BIT_NUM_REG stores the Remainder Bit Number, which indicates the number of effective bits of incomplete blocks in plaintext/ciphertext. The value stored in this register equals to length(P)/128 or length(C)/128. AES Accelerator only uses this register when working in the DMA-AES mode.

Register AES_REMAINDER_BIT_NUM_REG does not affect the results of plaintext or ciphertext, but does impact the value of T_0, therefore the Tag value too.

The GCM Algorithm can be viewed as the combination of GCTR operation and GHASH operation, among which, the GCTR performs the encryption and decryption, while the GHASH sloves the Tag.

Note that the AES_REMAINDER_BIT_NUM_REG register is only effective for GCM encryption. To be more specific:

- For GCM encryption, the Hardware firstly computes C, then passes it in the form of **TEXT-PADDING**(C) as the input of GHASH operation. In this case, hardware determines how many trailing "0" should be added based on the content of AES_REMAINDER_BIT_NUM_REG.
- For GCM decrpytion, the padding is completed with the **TEXT-PADDING**(C) function. In this case, the AES_REMAINDER_BIT_NUM_REG register is not effective.

25.8 Pseudo-Round Anti-DPA Function

The pseudo-round function of ESP32-P4's AES can achieve higher anti-attack performance. When the pseudo-round function is enabled, AES will randomly add pseudo-rounds before and after the original operation rounds. In a pseudo-round, AES will use the pseudo-key, which is automatically generated by the hardware, to perform a round of fake operations. The operations in the pseudo-round do not affect the original AES operation results.

The configuration related to pseudo-round is as follows:

Mode of Pseudo-Round Function

Users can choose to enable pseudo-round function by configuring the AES_PSEUDO_EN register:

- 0: Not enable pseudo-round function;
- 1: Enable pseudo-round function.

Total number of Pseudo-Round

The total number of pseudo-rounds that will be randomly inserted into each AES operation round is controlled by:

- pseudo_base: Equal to the value of AES_PSEUDO_BASE, which configures the basic number of pseudo-rounds.

- pseudo_inc: Equal to the value of AES_PSEUDO_INC, which configures the random incremental number of pseudo-rounds.

The total number of pseudo-rounds will be randomly in the range

$$[pseudo\_base, pseudo\_base + (2^{pseudo\_inc} - 1)]$$

Random Number Update Frequency

AES_PSEUDO_RNG_CNT configures the frequency of random key updates in the pseudo-round function. The higher the configured value, the higher the update frequency. This value is usually recommended to be set to maximum (7).

25.9 Interrupts

ESP32-P4's AES can generate the following interrupt signal(s) that will be sent to the Interrupt Matrix.

- AES_INTR

There is one internal interrupt source from AES that can generate the above interrupt signal(s). The interrupt source from AES are listed with its trigger condition and the resulted interrupt signal(s) in Table 25.9-1.

Table 25.9-1. AES's Internal Interrupt Sources

Internal Interrupt Source	Trigger Condition	Interrupt Signal
AES_DMA_DONE_INT	Completion of an AES DMA operation	AES_INTR

Note:
For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter 12 *Interrupt Matrix* > Section 12.2 *Interrupt Terminology in ESP32-P4*.

25.10 Memory Summary

The addresses in this section are relative to the AES accelerator base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Size (byte)	Starting Address	Ending Address	Access
AES_IV_MEM	Memory IV	16 bytes	0x0050	0x005F	R/W
AES_H_MEM	Memory H	16 bytes	0x0060	0x006F	RO
AES_J0_MEM	Memory J0	16 bytes	0x0070	0x007F	R/W
AES_T0_MEM	Memory T0	16 bytes	0x0080	0x008F	RO

25.11 Register Summary

The addresses in this section are relative to the AES accelerator base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

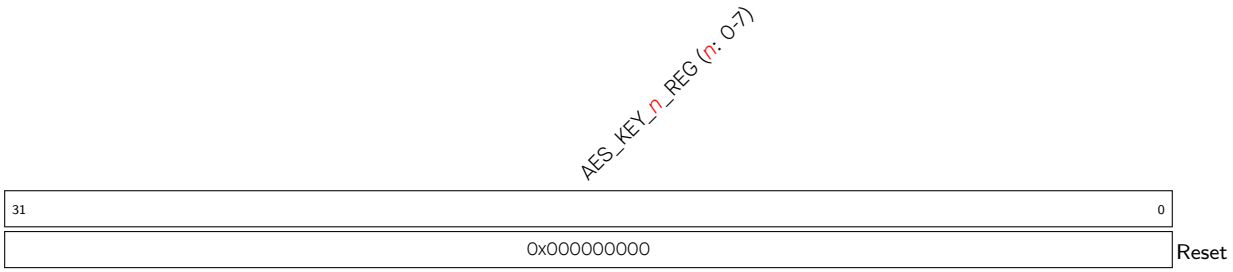
Name	Description	Address	Access
Key Registers			
AES_KEY_0_REG	AES key data register 0	0x0000	R/W
AES_KEY_1_REG	AES key data register 1	0x0004	R/W
AES_KEY_2_REG	AES key data register 2	0x0008	R/W
AES_KEY_3_REG	AES key data register 3	0x000C	R/W
AES_KEY_4_REG	AES key data register 4	0x0010	R/W
AES_KEY_5_REG	AES key data register 5	0x0014	R/W
AES_KEY_6_REG	AES key data register 6	0x0018	R/W
AES_KEY_7_REG	AES key data register 7	0x001C	R/W
TEXT_IN Registers			
AES_TEXT_IN_0_REG	Source text data register 0	0x0020	R/W
AES_TEXT_IN_1_REG	Source text data register 1	0x0024	R/W
AES_TEXT_IN_2_REG	Source text data register 2	0x0028	R/W
AES_TEXT_IN_3_REG	Source text data register 3	0x002C	R/W
TEXT_OUT Registers			
AES_TEXT_OUT_0_REG	Result text data register 0	0x0030	RO
AES_TEXT_OUT_1_REG	Result text data register 1	0x0034	RO
AES_TEXT_OUT_2_REG	Result text data register 2	0x0038	RO
AES_TEXT_OUT_3_REG	Result text data register 3	0x003C	RO
Control or Configuration Registers			
AES_MODE_REG	Defines key length and encryption/decryption	0x0040	R/W
AES_DMA_ENABLE_REG	Selects the working mode of the AES accelerator	0x0090	R/W
AES_BLOCK_MODE_REG	Defines the block cipher mode	0x0094	R/W
AES_BLOCK_NUM_REG	Block number configuration register	0x0098	R/W
AES_INC_SEL_REG	Standard incrementing function register	0x009C	R/W
AES_AAD_BLOCK_NUM_REG	AAD block number configuration register	0x00A0	R/W
AES_REMAINDER_BIT_NUM_REG	Remainder bit number of plaintext/ciphertext	0x00A4	R/W
AES_TRIGGER_REG	Operation start controlling register	0x0048	WT
AES_CONTINUE_OP_REG	Operation continue controlling register	0x00A8	WO
AES_DMA_EXIT_REG	Operation exit controlling register	0x00B8	WO
AES_PSEUDO_REG	Pseudo-round function configuration register	0x00D0	R/W
Status Register			
AES_STATE_REG	Operation status register	0x004C	RO
Interrupt Registers			
AES_INT_CLR_REG	DMA-AES interrupt clear register	0x00AC	WT
AES_INT_ENA_REG	DMA-AES interrupt enable register	0x00B0	R/W

25.12 Registers

The addresses in this section are relative to the AES accelerator base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

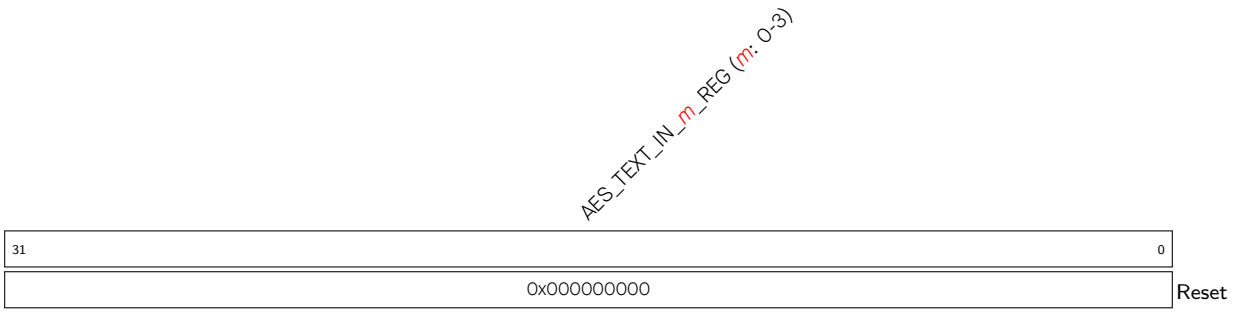
For how to program reserved fields, please refer to Section IX .

Register 25.1. AES_KEY_*n*_REG (*n*: 0-7) (0x0000+4*n)



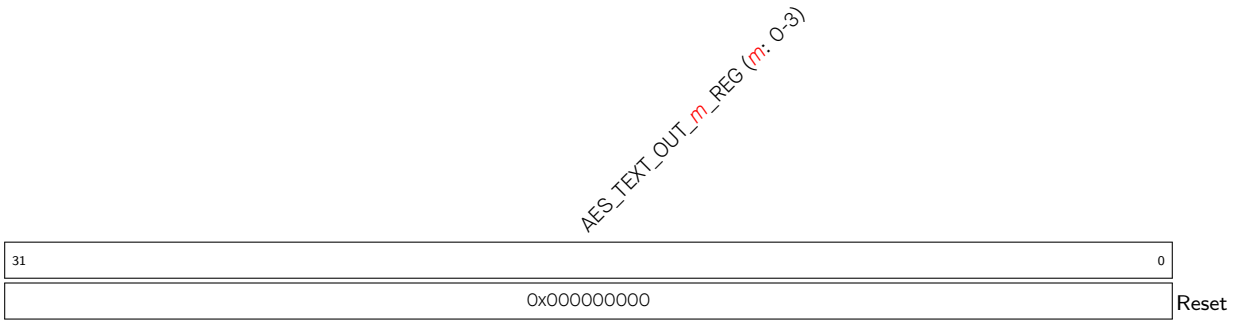
AES_KEY_*n*_REG (*n*: 0-7) Represents AES key data. (R/W)

Register 25.2. AES_TEXT_IN_*m*_REG (*m*: 0-3) (0x0020+4*m)



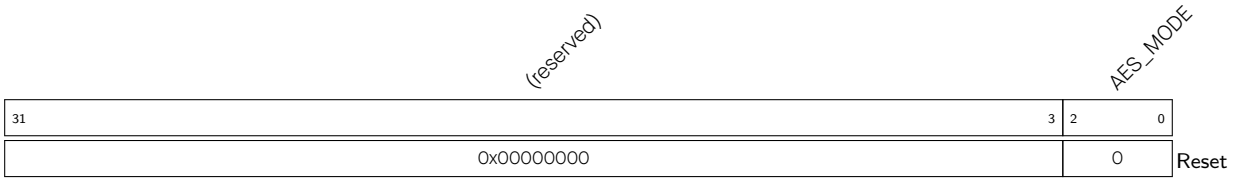
AES_TEXT_IN_*m*_REG (*m*: 0-3) Represents the source text data when the AES accelerator operates in the Typical AES working mode. (R/W)

Register 25.3. AES_TEXT_OUT_*m*_REG (*m*: 0-3) (0x0030+4*m)



AES_TEXT_OUT_*m*_REG (*m*: 0-3) Represents the result text data when the AES accelerator operates in the Typical AES working mode. (RO)

Register 25.4. AES_MODE_REG (0x0040)



AES_MODE Configures the key length and encryption/decryption of the AES accelerator.

0: AES-128 encryption

1: Reserved

2: AES-256 encryption

3: Reserved

4: AES-128 decryption

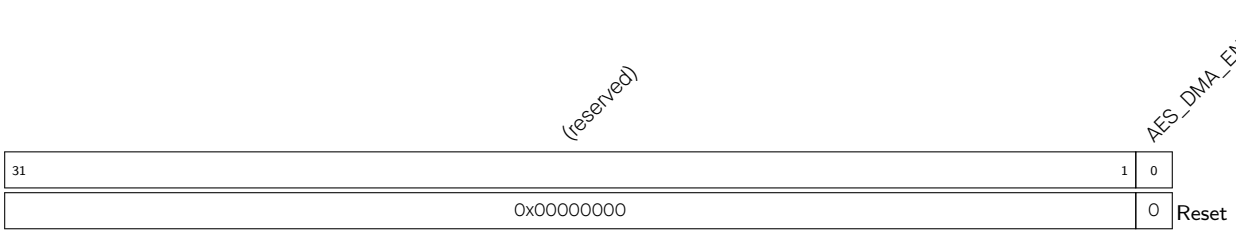
5: Reserved

6: AES-256 decryption

7: Reserved

(R/W)

Register 25.5. AES_DMA_ENABLE_REG (0x0090)



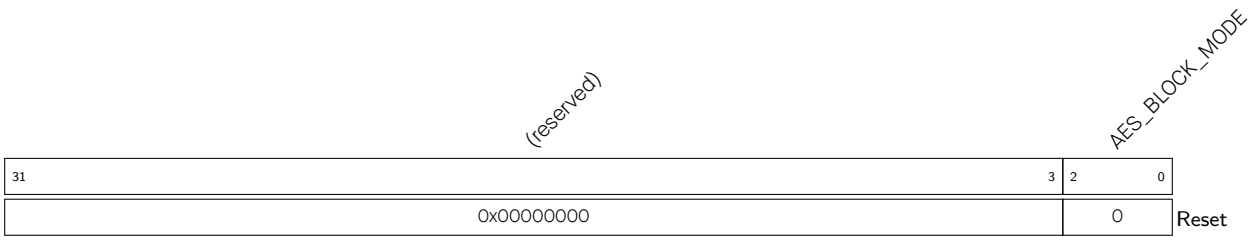
AES_DMA_ENABLE Configures the working mode of the AES accelerator.

0: Typical AES

1: DMA-AES

(R/W)

Register 25.6. AES_BLOCK_MODE_REG (0x0094)



AES_BLOCK_MODE Configures the block cipher mode of the AES accelerator operating under the DMA-AES working mode.

0: ECB (Electronic Code Block)

1: CBC (Cipher Block Chaining)

2: OFB (Output FeedBack)

3: CTR (Counter)

4: CFB8 (8-bit Cipher FeedBack)

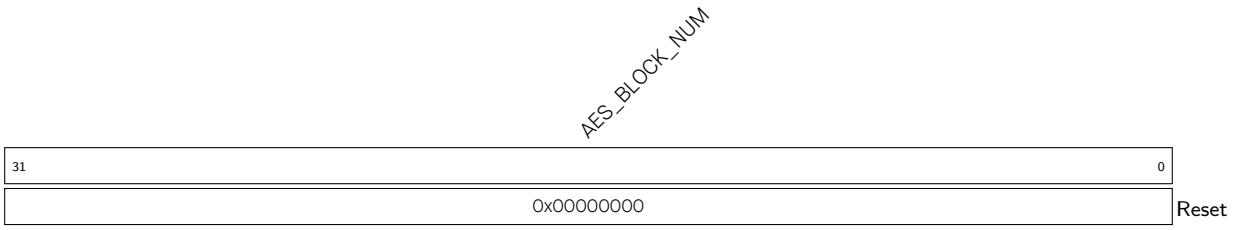
5: CFB128 (128-bit Cipher FeedBack)

6: GCM (Galois/Counter Mode)

7: Reserved

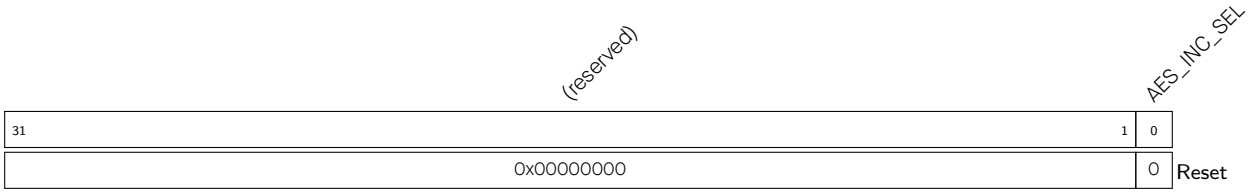
(R/W)

Register 25.7. AES_BLOCK_NUM_REG (0x0098)



AES_BLOCK_NUM Represents the Block Number of plaintext or ciphertext when the AES accelerator operates under the DMA-AES working mode. For details, see Section 25.6.4. (R/W)

Register 25.8. AES_INC_SEL_REG (0x009C)



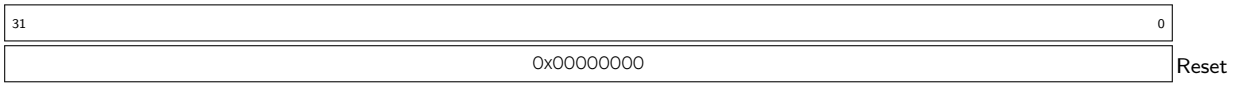
AES_INC_SEL Configures the Standard Incrementing Function for CTR block operation.

0: INC_{32}

1: INC_{128}

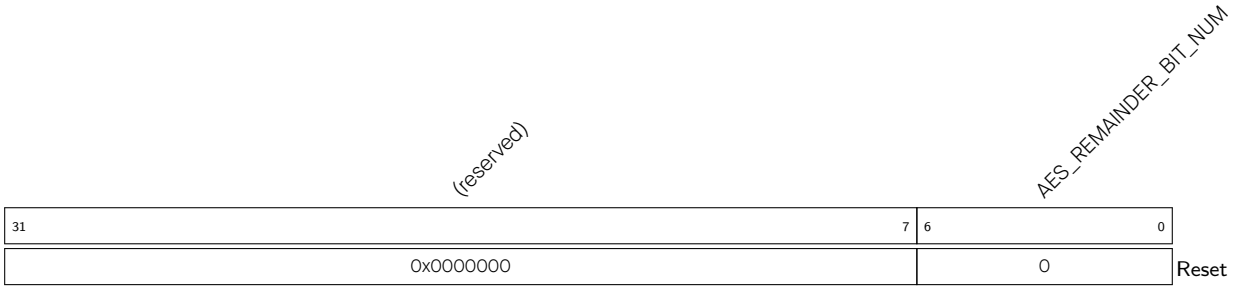
(R/W)

Register 25.9. AES_AAD_BLOCK_NUM_REG (0x00A0)



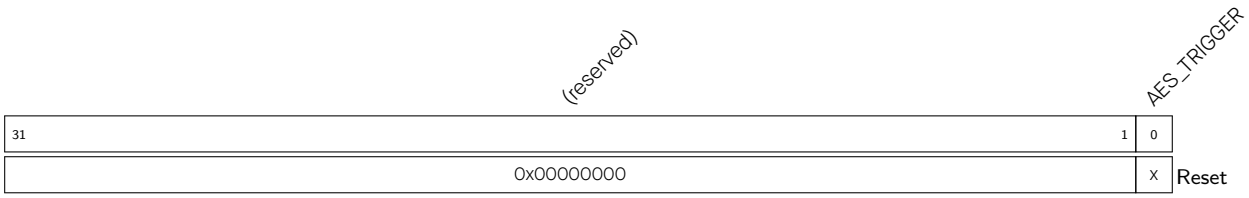
AES_AAD_BLOCK_NUM Stores the ADD block number for the GCM operation. For details, see Section 25.7.4. (R/W)

Register 25.10. AES_REMAINDER_BIT_NUM_REG (0x00A4)



AES_REMAINDER_BIT_NUM Stores the Remainder Bit Number for the GCM operation. For details, see Section 25.7.5. (R/W)

Register 25.11. AES_TRIGGER_REG (0x0048)



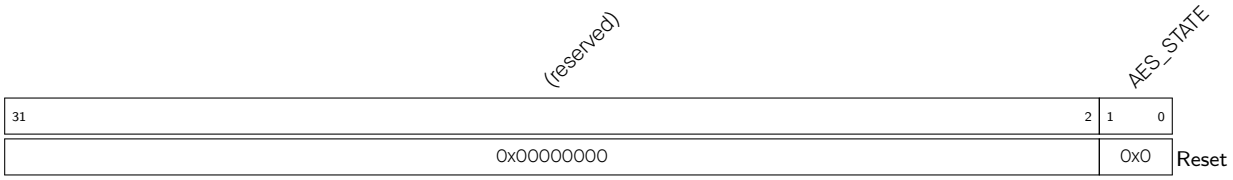
AES_TRIGGER Configures whether to start AES operation.

0: No effect

1: Start

(WT)

Register 25.12. AES_STATE_REG (0x004C)



AES_STATE Represents the working status of the AES accelerator.

In Typical AES working mode:

0: IDLE

1: WORK

2: No effect

3: No effect

In DMA-AES working mode:

0: IDLE

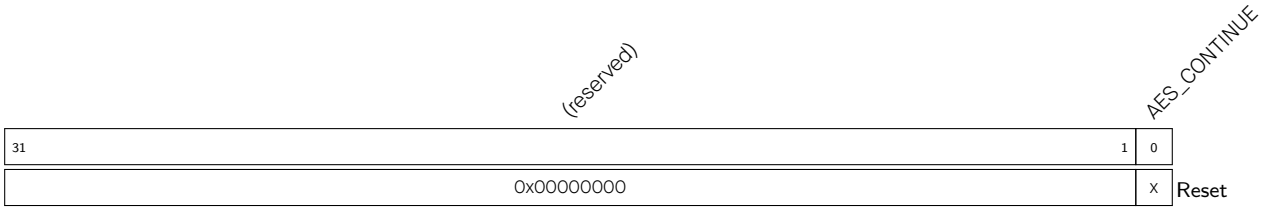
1: WORK

2: DONE

3: No effect

(RO)

Register 25.13. AES_CONTINUE_OP_REG (0x00A8)



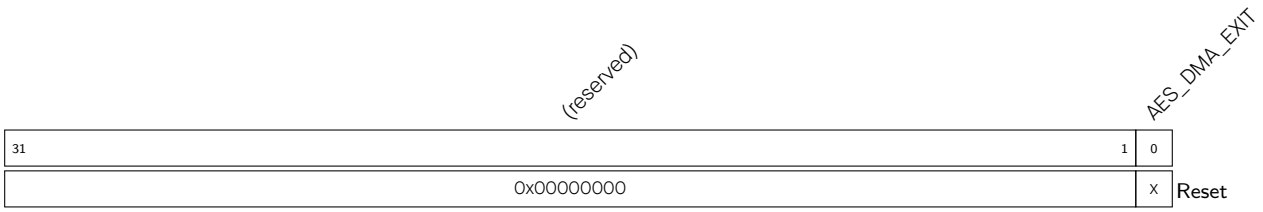
AES_CONTINUE_OP Configures whether to continue AES operation.

0: No effect

1: Continue

(WO)

Register 25.14. AES_DMA_EXIT_REG (0x00B8)



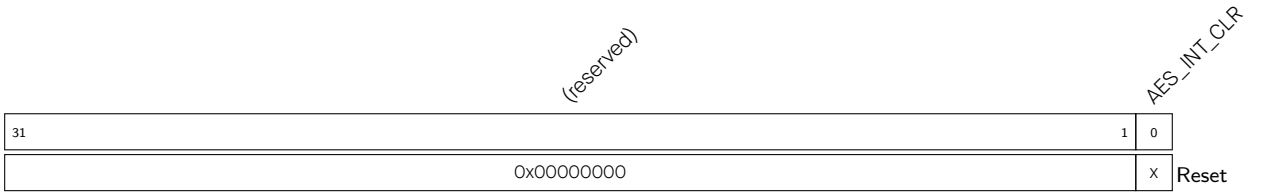
AES_DMA_EXIT Configures whether to exit AES operation.

0: No effect

1: Exit

Only valid for DMA-AES operation. (WO)

Register 25.15. AES_INT_CLR_REG (0x00AC)



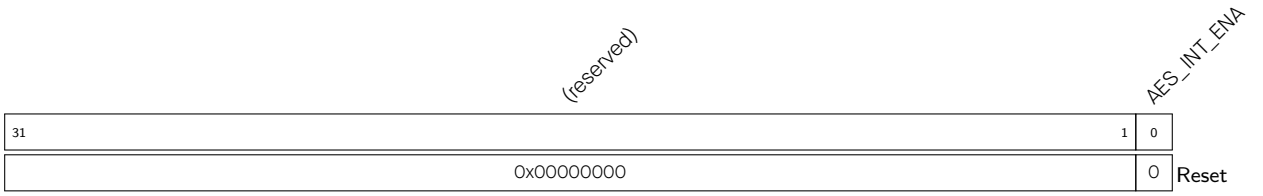
AES_INT_CLR Configures whether to clear AES interrupt.

0: No effect

1: Clear

(WT)

Register 25.16. AES_INT_ENA_REG (0x00B0)



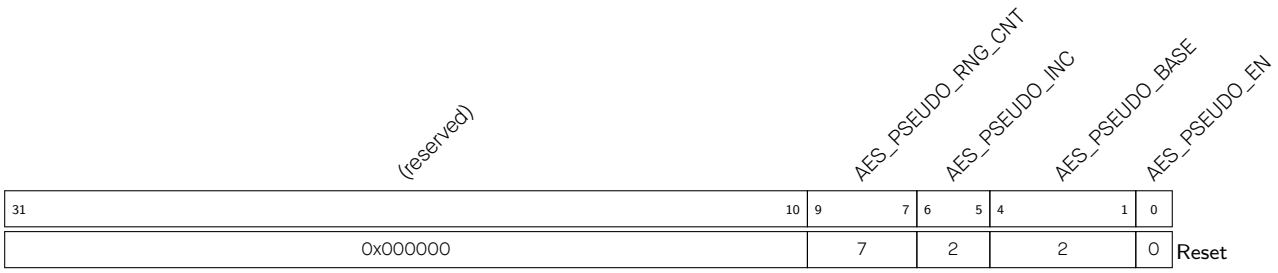
AES_INT_ENA Configures whether to enable AES interrupt.

0: Disable

1: Enable

(R/W)

Register 25.17. AES_PSEUDO_REG (0x00D0)



AES_PSEUDO_EN Configures whether to enable the pseudo-round function of AES.

0: Disable

1: Enable

(R/W)

AES_PSEUDO_BASE Configures the basic number of pseudo-rounds. (R/W)

AES_PSEUDO_INC Configures the random incremental number of pseudo-rounds. (R/W)

AES_PSEUDO_RNG_CNT Configures the frequency of pseudo-key updates in the pseudo-round function. (R/W)